MCI lab 11

Haider Zaidi (mz10270), Ali Hussain (sz10384)

Task 1:

Code:

```
volatile uint8_t timer_flag = 0;
/* USER CODE END PV */

/* Private function prototypes ---------
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_SPI1_Init(void);
static void MX_TIM2_Init(void);
static void MX_USART1_UART_Init(void);
static void MX_USB_PCD_Init(void);
```

```
MX_GPIO_Init();
MX_I2C1_Init();
MX_SPI1_Init();
MX_TIM2_Init();
MX_USART1_UART_Init();
MX_USB_PCD_Init();

/* USER CODE BEGIN 2 */
IMU_Init(&hi2c1, &hspi1, &imu);       // Ini
IMU_Offset(&hi2c1, &hspi1, &imu);     // Cal
HAL_TIM_Base_Start_IT(&htim2);        // Sta
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
```

```
while (1)
{
  if (timer_flag)
  {
    timer_flag = 0;
    IMU_Read(&hi2c1, &hspi1, &imu);   // Read sensors + apply complementary filter
    IMU_Print(&huart1, &imu);         // Transmit: acc_angle, gyro_x, filtered_angle
    //HAL_Delay(100);
  }
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI|RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_BYPASS;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL6;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }
```

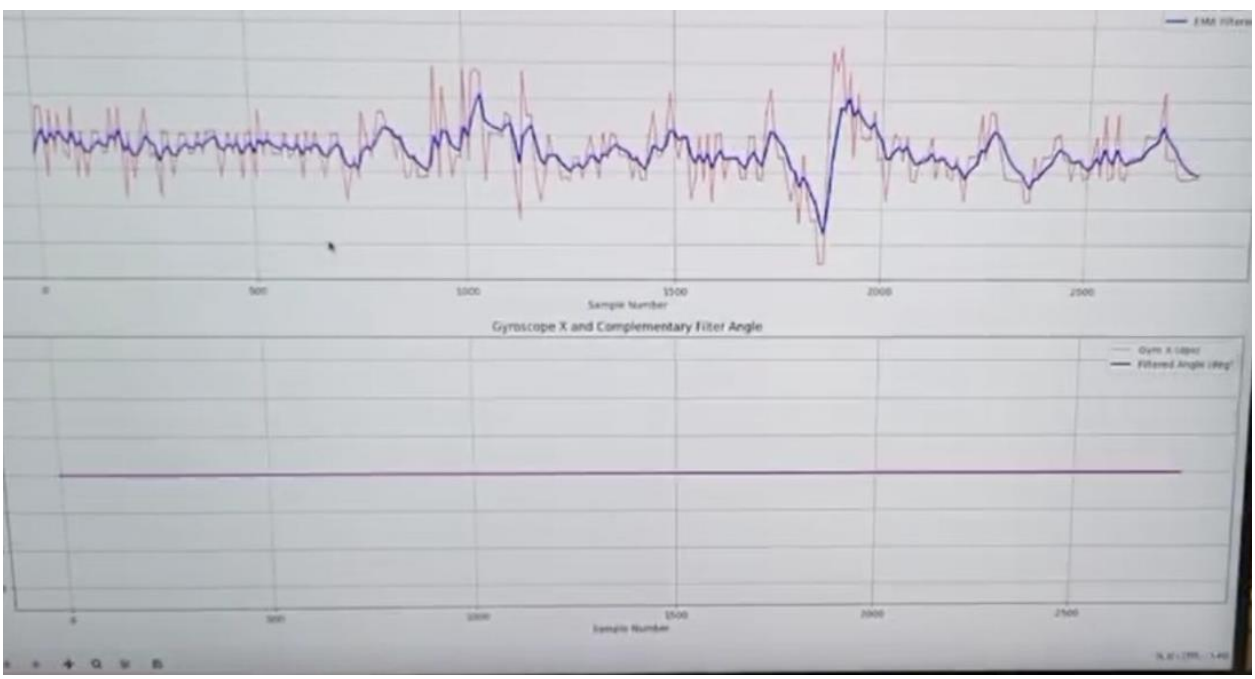
Code: (Math plot)

```python
import serial
import matplotlib.pyplot as plt
from collections import deque

# === Setup Serial ===
ser = serial.Serial('/dev/ttyUSB0', 115200)
plt.ion()

# Data buffers
accX_vals = deque(maxlen=500)
gyroX_vals = deque(maxlen=500)
angle_vals = deque(maxlen=500)
filtered_accX_vals = deque(maxlen=500)
time_ms = deque(maxlen=500)
cnt = 0

# Filter parameters
ALPHA = 0.3  # Smoothing factor (0-1), lower = more smoothing
filtered_acc = 0  # Initial filtered value

# === Plotting Function ===
def makeFig():
    plt.subplot(2, 1, 1)
    plt.cla()
    plt.title('Accelerometer X - Raw vs Exponential Moving Average')
    plt.grid(True)
    plt.xlabel('Sample Number')
    plt.ylabel('Acceleration (g)')
```

```python
    if len(gyroX_vals) > 0:
        plt.plot(list(time_ms), list(gyroX_vals), 'g.-', label='Gyro X (dps)', alpha=0.4, markersize=2)
    if len(angle_vals) > 0:
        plt.plot(list(time_ms), list(angle_vals), 'm-', label='Filtered Angle (deg)', linewidth=2)

    plt.legend(loc='upper right')
    plt.ylim(-180, 180)
    plt.tight_layout()

# === Main Loop ===
print("Starting plot with Exponential Moving Average filter (Alpha = 0.3)")
print("Graph 1: Raw (red) vs Filtered (blue) Accelerometer X")
print("Graph 2: Gyroscope X (green) and Filtered Angle (magenta)")
```

```python
while True:
    while ser.inWaiting() == 0:
        pass

    try:
        line = ser.readline().decode().strip()
        values = line.split(',')

        if len(values) == 3:
            accX = float(values[0])
            gyroX = float(values[1])
            angle = float(values[2])

            # Apply Exponential Moving Average (EMA) filter
            if cnt == 0:
                filtered_acc = accX  # Initialize on first sample
            else:
                filtered_acc = ALPHA * accX + (1 - ALPHA) * filtered_acc

            # Store values
            accX_vals.append(accX)
            filtered_accX_vals.append(filtered_acc)
            gyroX_vals.append(gyroX)
            angle_vals.append(angle)
            time_ms.append(cnt)
            cnt += 10

            makeFig()
            plt.pause(0.0001)
```

Output:



Task 2:

Code

```
#define SETPOINT 0.0f

/* ----------------------------------
 * PWM Output Clamping
 * TIM3 period = 999, so 999 = 100% du
 * ----------------------------------
#define PID_OUT_MAX  999.0f
#define PID_OUT_MIN -999.0f

/* ----------------------------------
 * Anti-Windup: Clamp integral accumul
 * ----------------------------------
#define INTEGRAL_MAX  200.0f
#define INTEGRAL_MIN -200.0f

/* ----------------------------------
 * Complementary Filter Blend Ratios
 * 98% gyro (fast response) + 2% acce
 * ----------------------------------
#define COMP_GYRO_WEIGHT  0.98f
#define COMP_ACC_WEIGHT   0.02f
```

```
/* Private function prototypes -------------
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_SPI1_Init(void);
static void MX_TIM3_Init(void);
static void MX_USB_PCD_Init(void);
static void MX_I2C2_Init(void);
static void MX_TIM2_Init(void);
static void MX_USART2_UART_Init(void);
```

```
void I3G_WriteReg(uint8_t reg_addr, uint8_t data) {
    uint8_t tx_buffer[2];
    tx_buffer[0] = reg_addr & 0x7F;  // Bit 7 = 0 for WRITE
    tx_buffer[1] = data;

    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, tx_buffer, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);
}

uint8_t I3G_ReadReg(uint8_t reg_addr) {
    uint8_t tx_data = reg_addr | 0x80;  // Bit 7 = 1 for READ
    uint8_t rx_data;

    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);
    HAL_SPI_Transmit(&hspi1, &tx_data, 1, HAL_MAX_DELAY);
    HAL_SPI_Receive(&hspi1, &rx_data, 1, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);

    return rx_data;
}
```

```c
void Motor_Left_Forward(uint16_t speed) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_RESET);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, speed);
}

void Motor_Left_Backward(uint16_t speed) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, speed);
}

void Motor_Left_Stop(void) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_7, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_6, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_2, 0);
}
```

```c
void Motor_Right_Forward(uint16_t speed) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_RESET);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, speed);
}

void Motor_Right_Backward(uint16_t speed) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, speed);
}

void Motor_Right_Stop(void) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_8, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_12, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 0);
}
```

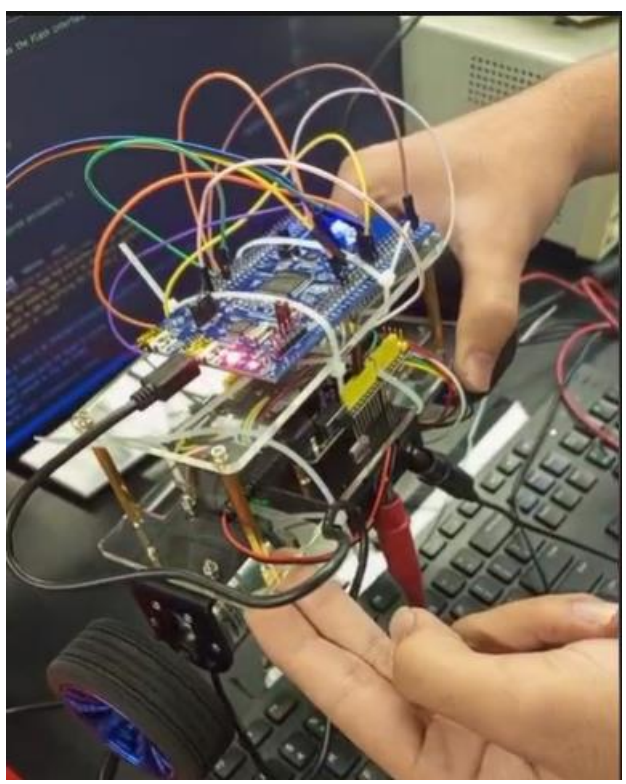
```c
void Motors_Drive(float pid_output) {
    uint16_t pwm_val;

    if (pid_output > 0) {
        if (pid_output > PID_OUT_MAX) pid_output = PID_OUT_MAX;
        pwm_val = (uint16_t)pid_output;
        Motor_Left_Forward(pwm_val);
        Motor_Right_Forward(pwm_val);
    }
    else if (pid_output < 0) {
        float abs_out = -pid_output;
        if (abs_out > PID_OUT_MAX) abs_out = PID_OUT_MAX;
        pwm_val = (uint16_t)abs_out;
        Motor_Left_Backward(pwm_val);
        Motor_Right_Backward(pwm_val);
    }
    else {
        Motor_Left_Stop();
        Motor_Right_Stop();
    }
}
```

```c
while (1)
{
    /* Print debug data at ~10Hz (throttled by ISR via display_flag) */
    if (display_flag == 1) {
        display_flag = 0;

        /* No float format specifier in nano-printf — split into integer + fractional parts */
        int shared_angle_i = (int)shared_angle;
        int shared_angle_f = (int)(fabsf(shared_angle - shared_angle_i) * 100);

        int acc_angle_i = (int)acc_angle;
        int acc_angle_f = (int)(fabsf(acc_angle - acc_angle_i) * 100);

        int gyro_rate_i = (int)gyro_rate;
        int gyro_rate_f = (int)(fabsf(gyro_rate - gyro_rate_i) * 100);

        int shared_pid_out_i = (int)shared_pid_out;
        int shared_pid_out_f = (int)(fabsf(shared_pid_out - shared_pid_out_i) * 100);

        cout("Angle: %d.%02d\r\nAcc: %d.%02d\r\nGyro: %d.%02d\r\nPID: %d.%02d\r\n",shared_angle_i, shared_angle_f,acc_angle_i, acc_angle_f,gyro_rate_i, gyro_rate_f,shared_pid_out_i, shared_pid_out_f);
    }
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

```c
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {

    if (htim->Instance == TIM2) {

        static float tilt_angle   = 0.0f;
        static float integral     = 0.0f;
        static float prev_error   = 0.0f;
        static int   uart_counter = 0;

        /* Step 1 — Gyroscope: read Y-axis, convert raw counts to °/s
           Sensitivity = 250dps range → 0.00875 °/s per LSB */
        uint8_t y_low  = I3G_ReadReg(0x2A);  // OUT_Y_L
        uint8_t y_high = I3G_ReadReg(0x2B);  // OUT_Y_H
        int16_t raw_gyro_y = (int16_t)((y_high << 8) | y_low);
        float gyro_y_dps = (float)raw_gyro_y * 0.00875f;

        /* Step 2 — Accelerometer: burst-read 6 bytes starting at OUT_X_L_A (0x28)
           MSB flag (0x80) enables auto-increment on LSM303DLHC */
        uint8_t acc_reg = 0x28 | 0x80;
        uint8_t acc_buf[6] = {0};
        HAL_I2C_Master_Transmit(&hi2c1, 0x32, &acc_reg, 1, 10);
        HAL_I2C_Master_Receive(&hi2c1, 0x33, acc_buf, 6, 10);
        int16_t acc_raw_x = (int16_t)((acc_buf[1] << 8) | acc_buf[0]);
        int16_t acc_raw_z = (int16_t)((acc_buf[5] << 8) | acc_buf[4]);
        float acc_angle_deg = atan2f((float)acc_raw_x, (float)acc_raw_z) * (180.0f / 3.14159f);

        /* Step 3 — Complementary filter: blend gyro integration with accelerometer
           Gyro tracks fast motion; accelerometer corrects long-term drift */
        tilt_angle = COMP_GYRO_WEIGHT * (tilt_angle + gyro_y_dps * DT)
                   + COMP_ACC_WEIGHT  * acc_angle_deg;

        /* Step 4 — PID controller */
        float error = SETPOINT - tilt_angle;
```

```
    float p_term = KP * error;

    integral += KI * error * DT;
    if (integral > INTEGRAL_MAX) integral = INTEGRAL_MAX;    // Anti-windup clamp
    if (integral < INTEGRAL_MIN) integral = INTEGRAL_MIN;

    float derivative = (error - prev_error) / DT;
    float d_term = KD * derivative;
    prev_error = error;

    float pid_output = p_term + integral + d_term;

    if (pid_output > PID_OUT_MAX) pid_output = PID_OUT_MAX;  // Output clamp
    if (pid_output < PID_OUT_MIN) pid_output = PID_OUT_MIN;

    /* Step 5 — Send PID output to motors */
    Motors_Drive(pid_output);

    /* Step 6 — Publish to shared variables (read by main loop display) */
    shared_angle   = tilt_angle;
    shared_pid_out = pid_output;
    gyro_rate      = gyro_y_dps;
    acc_angle      = acc_angle_deg;

    /* Step 7 — Flag main loop to print every 20 ticks = 10Hz */
    if (++uart_counter >= 20) {
        display_flag = 1;
        uart_counter = 0;
    }
}
```

Output:



Evaluation Questions

What role does each PID term play in the controller's response?

The proportional (P) term responds to the current error by applying a corrective force proportional to its magnitude, providing an immediate but often steady-state offset. The integral (I) term accumulates past errors over time to eliminate this offset, ensuring the output reaches the setpoint, though it can introduce overshoot or instability if too strong. The derivative (D) term anticipates future error based on its rate of change, dampening oscillations and improving stability by counteracting sudden changes, but it is sensitive to measurement noise.

How does changing Kp, Ki, or Kd affect system performance?

Increasing Kp makes the system faster and reduces steady-state error but can cause overshoot and instability if too high. Raising Ki eliminates steady-state error more aggressively but adds overshoot, oscillations, and longer settling time if excessive. Increasing Kd adds damping, reducing overshoot and improving stability, but excessive Kd amplifies noise and can slow the response.

What challenges did you face in initial tuning the controller? What will be your strategy for the final fine tuning of the controller?

During initial tuning, the main challenge was the gyroscope's slow response, making it difficult to determine suitable Kp, Kd, Ki values. With no clear starting point, we resorted to trial and error, randomly testing numbers between 1 and 80 using a binary selection approach. For final fine-tuning, the strategy will be systematic: first set Ki and Kd to zero, then increase Kp until the system oscillates steadily, next add Kd to dampen overshoot, and finally introduce small Ki to eliminate steady-state error, adjusting each incrementally based on step response observations.

What are the limitations of using a simple PID for real-world systems?

A simple PID controller struggles with nonlinearities, noise sensitivity (especially derivative gain), time delays, and varying system dynamics. It also suffers from integral windup during actuator saturation and cannot predict future errors, limiting performance in complex, real-world environments.